

Climbing to the Frontier — METHODOLOGY

BAINSA Hackathon, Track 3 (Anthropic), 26 September 2026 — Team: Lorenzo Pazienza (lead), Seby, Teo, Niccolò

1. Game plan

- **Triage first.** Before writing anything we read all 24 cells, checked the literature for each problem (Griggs–Ho for P3, O’Bryant and Fornal–Sun for P4, Bilyk–Matzke and Fodor–Víg–Zarnócz for P1), and ranked the cells by points against feasibility. The cheap cells were closed first. The time-boxed effort went to the C5 cells, which carry the weight, and partial C6 work came last.
- **One problem per person, one agent per sub-task.** Each team member owned a problem and ran Claude Code (Opus) sessions for it. The lead ran several further sessions (P2 search, P4 cross-check, P4 C6, and a finaliser that assembles the hand-in), Codex as an independent second model, and a Claude coordinator that tracked the state of every cell. A single status table (cell $\rightarrow$ file $\rightarrow$ state $\rightarrow$ owner) was the source of truth, and the lead kept the plan and made every accept/reject decision.
- **Computation only where the rules allow it.** We used the computer to explore, and relied on it in a proof only over a finite set that our own argument had reduced to, with code shipped and runtimes stated. Every computational cell follows the five-point exhaustiveness certificate.

2. Correctness: nothing is accepted on one model’s word

- **Exact evaluators outside the model.** Scores such as the uphill-path counts come from a small evaluator with two independent counting methods, a DP and brute-force enumeration. No number in a write-up comes from the model’s own arithmetic.
- **Two independent implementations for every certificate.** In P4 the SAT pipeline (Codex) and the DFS pipeline (Teo) were written separately. Their survivor lists agree element by element, with the same SHA-256, for all $k \leq 16$. In P2 the SAT lemmas were run with CaDiCaL and Glucose, and re-implemented from scratch by a different agent that shared no code with the first. Sanity runs at the feasible sizes show the encodings are not vacuous.
- **Adversarial review by a different agent than the author.** Examples:
 - P1 C5 (the cycle lemma) was reviewed line by line by Codex and by Claude, with numerical stress tests (40k random cycles, 10^6 points for the key inequality).
 - P3 C4/C5 were re-verified by an independent exhaustive checker that finds cycles straight from the definition, without assuming Brandt’s theorem, up to $k = 12$, beyond the authors’ $n \leq 60$.
 - P2’s new lower-bound proof was checked step by step, including recomputing the Delsarte table.
 - A final read-only referee pass (Codex) covered all cells before hand-in. It found packaging and labelling problems (labelling-file format, code not attached to the upload, a “by hand” label on a computer-assisted step, missing certificate data for P4), and we fixed them before submitting.
- **Honest labels.** A claim is *solved* only with a complete proof. Everything else is *partial*, with the proved part stated. Every result that depends on a computer says *computer-assisted*, and unfinished runs are reported as unfinished. For example, we claim the certified boundary $K^* = 14$ rather than 15 or 16: every survivor up to $k = 16$ is decided, but the timed rerun at $k = 15$ took 613s, just over the 10-minute limit in the rules.

3. Efficiency (compute and tokens)

- **Measure before waiting.** We estimated the runtime of long jobs from uniform random samples of their inputs. That showed a $k = 44$ certificate needs about 240 CPU-hours and $k = 48$ even more, so we stopped both and kept the (computer-assisted, 4-second) proof of $k = 32, 38, 42$.
- **Stop duplicates early.** When two sessions were found working on the same cell (P1 C3; the $k = 32$ case of P4 C6), one was stopped and redirected.
- **Short, targeted prompts to agents.** Agents received the official cell text, the exact files to read and a deadline, and wrote into their own folders. Only the finaliser wrote into the hand-in folder. This avoided re-reading large contexts and conflicting edits.

4. Human verification and intervention

- The lead read every agent report and decided what to accept, what to send for review, and what to stop. Several agent claims were corrected before hand-in, for instance an over-claimed status on P2 and a mislabelled “solved” on P3 C3/C5, which became *partial*.

- Where an agent had paraphrased a cell statement, we replaced it with the official text from the problem page (for example P4 C4/C5), and we relabelled cells whose write-up claimed more than it proved.
- When a teammate's independent work turned out stronger than ours (the P2 lower bound via decycling sets), we switched the hand-in to it after verifying it, rather than keeping our own.

5. Hand-in and reproducibility

Everything is in <https://github.com/lorenzopazienza/HACKATLON> (folder `BAINSA_team_submission/`): one folder per cell with the PDF, the $\text{\LaTeX}$ source, the code and the logs of every computation used. Code used in a proof runs in under 10 minutes on a laptop, and its timed log is included. On the portal, each cell carries the full argument as text and a link to its folder.

6. What went wrong, and the fix

Around midday several agents and people were working on the same cells, from four copies of the same folder. We fixed this with three rules: one owner per cell, one hand-in folder written by one session only, and a live status table. From then on no cell had two writers.